# Context Sensitivity in Java Pointer Analysis

Malay Kedia

January to April 2026

## Contents

**1 Introduction** **2**

**2 Background** **2**
2.1 Andersen's analysis . . . 2
2.2 Call Site sensitivity and $k$-CFA . . . 3

**3 Object Sensitivity** **4**
3.1 Why call site sensitivity is incomplete for Java . . . 4
3.2 Receiver objects as contexts . . . 5
3.3 Parameterization . . . 5
3.4 Constraint rules . . . 5
3.5 Incomparability with call site sensitivity . . . 6

**4 Picking Contexts** **6**
4.1 RECORD and MERGE . . . 6
4.2 Type sensitivity . . . 7
4.3 Context as a classifier . . . 8
4.4 Taxonomy . . . 8

**5 Selective Context Sensitivity** **8**
5.1 Three precision-loss patterns . . . 8
5.2 Object Flow Graph and Precision Flow Graph . . . 10
5.3 Algorithm . . . 10

**6 LSRV-contexts** **11**
6.1 Value-contexts . . . 11
6.2 Why value-contexts do not scale . . . 11
6.3 Compare Less . . . 11
6.4 Defer More . . . 12
6.5 The three-phase pipeline . . . 12

**7 Mixing Contexts** **12**
7.1 The premise . . . 12
7.2 LSRV with heap cloning . . . 13
7.3 The mixed abstraction `lsrvkobjH` . . . 13
7.4 The receiver-store optimisation . . . 13
7.5 The precision lattice . . . 13

# 1 Introduction

Pointer analysis answers a simple sounding question: for each reference variable in the program, what set of heap objects can it point to at run time? In Java, almost all values are references and almost all calls are virtual, so even something as basic as building the call graph requires knowing the runtime type of the receiver, which in turn requires knowing what the receiver can point to. This is why pointer analysis is the foundation of most of the other static analyses for Java. Pointer Analysis is essential for type inference, call-graph construction and for a lot of optimisations like scalar-replacement and stack allocation, devirtualisation etc.

The problem is undecidable in the general case. However, there are useful approximate answers. Every practical pointer analysis sits somewhere on a spectrum between two extremes: on one end, a fast but imprecise analysis that treats the entire program uniformly and merges a lot of information that should have been kept apart; and on the other end, a very precise analysis that distinguishes program states finely but takes too long to finish or runs out of memory on real programs.

The single biggest knob along this spectrum is *context sensitivity*. The idea is to analyse each method more than once, under different abstract contexts, so that information from one caller does not pollute the analysis of another. The papers in this reading trail differ in what they use as a context, where in the program they apply context sensitivity, and how they combine different kinds of context.

The rest of this report is organised in roughly chronological order. Section 2 sets up the background on Andersen's analysis and $k$-CFA. Section 3 covers the foundational paper on object sensitivity (Milanova et al., 2005). Section 4 covers the paper (Smaragdakis et al., 2011) that turned context choice into a principled design problem and introduced type sensitivity. Section 5 covers Li et al. (2018), which made context sensitivity selective. Section 6 covers Thakur and Nandivada (2019), which introduces LSRV-contexts as a scalable form of call-site sensitivity. Section 7 covers Thakur and Nandivada (2020), which mixes object and call site sensitivity.

# 2 Background

Before getting into the papers themselves, we fix a bit of vocabulary used. The two ideas that almost every paper in this report builds on are Andersen's analysis and $k$-CFA (call site sensitivity). Both are briefly summarised here.

## 2.1 Andersen's analysis

Andersen's analysis is the standard inclusion-based points-to analysis. It is context insensitive, flow insensitive, and field sensitive. The idea is to model the program as a set of subset constraints over points-to sets, and then compute the least solution.

For each variable $v$, the analysis computes a set $pt(v)$ of abstract heap objects that $v$ may point to at run time. Each `new` statement is named by an allocation site, and that site stands in for every concrete object the statement may create. Fields are tracked with a relation $pt(h, f)$, the set of objects that field $f$ of abstract object $h$ may point to. The four core rules are:

$$
\begin{aligned}
v_2 = \texttt{new}\ T \text{ at site } s &\implies o_s \in pt(v_2) \\
v_2 = v_1 &\implies pt(v_1) \subseteq pt(v_2) \\
v_2 = v_1.f &\implies \forall h \in pt(v_1).\ pt(h, f) \subseteq pt(v_2) \\
v_1.f = v_2 &\implies \forall h \in pt(v_1).\ pt(v_2) \subseteq pt(h, f)
\end{aligned}
$$

Virtual calls need a separate rule, because the dispatch target is not known until the receiver's points-to set has been computed. For a call $l = r_0.m(r_1, \ldots, r_n)$, let $\textsc{lookup}(T, m)$ denote the implementation of method $m$ that would be invoked on an object of dynamic type $T$. Then for every $h \in pt(r_0)$ the analysis resolves the call to the method $m' = \textsc{lookup}(\mathit{type}(h), m)$, marks $m'$ reachable, and adds the following constraints:

$$\begin{aligned} h &\in pt(\mathit{this}_{m'}) \\ pt(r_i) &\subseteq pt(p_i^{m'}) \quad \text{for each } i = 1, \ldots, n \\ pt(\mathit{ret}_{m'}) &\subseteq pt(l) \end{aligned}$$

where $p_i^{m'}$ is the $i$-th formal parameter of $m'$ and $\mathit{ret}_{m'}$ is its return variable. Because $pt(r_0)$ can grow during the fixed-point iteration, new dispatch targets and new constraint edges keep getting added until the analysis stabilises.

The analysis is fast and scales well, but it merges a lot of information that should have been kept apart. Every method is analysed exactly once. If the same method is called from two different callers with different arguments, the points-to sets of the parameters become the union of what each caller passed. Any context sensitive extension of the analysis is, at root, about keeping those two callers separated.

## 2.2 Call Site sensitivity and $k$-CFA

$k$-CFA is the standard recipe for adding context sensitivity. The context of a method analysis is the sequence of the last $k$ call sites on the stack. So under 1-CFA, a method called from two different call sites is analysed twice, once for each call site. Under 2-CFA, the context is the pair of the last two call sites, and so on. Setting $k = 0$ collapses to context insensitivity.

Consider an example.

```
Object id(Object x) { return x; }

void main() {
    Object a = new A();     // oA
    Object b = new B();     // oB
    Object p = id(a);       // call site c1
    Object q = id(b);       // call site c2
}
```

Under Andersen's analysis there is one summary of `id`. The parameter `x` ends up with $pt(x) = \{\texttt{oA}, \texttt{oB}\}$, the return value carries the same set, and so $pt(p)$ and $pt(q)$ are both $\{\texttt{oA}, \texttt{oB}\}$. Under 1-CFA, `id` is analysed twice. In context $c_1$, $pt(x^{c_1}) = \{\texttt{oA}\}$, so the call returns `oA` into `p`. In context $c_2$, $pt(x^{c_2}) = \{\texttt{oB}\}$, so the call returns `oB` into `q`. The two callers are kept apart and the result comes out as intended.

The motivation is straightforward: by keeping callers separate along the call stack, the analysis avoids the merge that Andersen's analysis suffers from. For functional and procedural languages, $k$-CFA fits well, because control flow there mostly follows the call structure.

For Java, however, $k$-CFA turns out to be both expensive and weak. The kind of imprecision $k$-CFA fixes is the same method called with different arguments from different call sites, which it solves by giving each call site its own context. The kind of imprecision that hurts in Java is two different receiver objects sharing a single summary of the same instance method, so their fields get merged in the field-store rule. The merge is keyed on the abstract receiver, not on the call stack, and it happens whenever the call to the instance method sits inside a wrapper, helper, or constructor that all receivers go through. 1-CFA only sees the last call site, so it cannot tell two

such receivers apart. 2-CFA fixes one wrapper layer; one more layer needs 3-CFA, and so on, with no fixed $k$ that handles arbitrary wrapper depth, and the cost growing fast in $k$. This is the gap that object sensitivity, the topic of the next section, was designed to close.

# 3 Object Sensitivity

The first paper in the trail is the parameterized object sensitivity work of Milanova et al. (2005). This is the paper that introduced object sensitivity in its current form.

## 3.1 Why call site sensitivity is incomplete for Java

The paper opens by listing three patterns where a context insensitive Andersen analysis loses precision, and showing that $k$-CFA does not really fix any of them. All three patterns trace back to the same root cause: a context insensitive analysis cannot tell that the receiver `this` of an instance method differs across distinct calls.

**Encapsulation.** A simple setter, called on different receivers with different arguments, ends up merging both stores in the single context insensitive copy of the method.

```
class Box {
    Object f;
    void set(Object x) { this.f = x; }
}

void main() {
    Box b1 = new Box();    // o1
    Box b2 = new Box();    // o2
    Object a = new A();    // oA
    Object b = new B();    // oB
    b1.set(a);
    b2.set(b);
}
```

What we want the analysis to conclude is that `b1.f` points to `oA` and `b2.f` points to `oB`. What a context insensitive analysis actually concludes is that both `b1.f` and `b2.f` point to the set $\{\texttt{oA}, \texttt{oB}\}$, because there is only one summary of `set` and it sees both stores.

While 1-CFA will be able to distinguish the contexts in this example, it will still have trouble with such patterns once the calls to `set` sit inside a wrapper, factory, or shared helper, because then both calls share the same call site of `set` and 1-CFA collapses them.

**Inheritance.** A field declared in a superclass and written by the superclass constructor is shared in the analysis across instances of all subclasses.

```
class A { Object f; A(Object x) { this.f = x; } }
class B extends A { B(Object x) { super(x); } }
class C extends A { C(Object x) { super(x); } }

B b = new B(new X());    // oB, oX
C c = new C(new Y());    // oC, oY
```

The intended result is that `b.f` points to `oX` and `c.f` points to `oY`. The actual result of a context insensitive analysis is that both `b.f` and `c.f` point to $\{\texttt{oX}, \texttt{oY}\}$, because the superclass constructor

`A(_)` has a single summary that absorbs both stores. Any client of this points-to information, say a virtual call resolver, then loses precision downstream.

**Containers.** A single allocation site for a container's internal storage causes objects placed in distinct containers to appear shared.

```
class Container {
    Object[] data = new Object[16];    // oArr
    void put(int i, Object o) { data[i] = o; }
}

Container c1 = new Container();    // o1
Container c2 = new Container();    // o2
c1.put(0, new X());                // oX
c2.put(0, new Y());                // oY
```

There is only one allocation site for the array, so a context insensitive analysis says both `c1.data` and `c2.data` point to the same `oArr`, and the array contents are merged into $\{\texttt{oX}, \texttt{oY}\}$. The intended result is that the array of `c1` contains only `oX` and the array of `c2` contains only `oY`.

### 3.2 Receiver objects as contexts

The fix proposed by Milanova et al. (2005) is to use the receiver allocation site as the context. Every instance method or constructor $m$ is analysed once for every abstract receiver $o$ on which $m$ may be called. Local variables and `this` are replicated, so each context has its own copy of the method's internal state.

The paper also makes the heap context dimension explicit. An object created by allocation site $s_q$ inside an instance method is named by appending $s_q$ to the receiver-name of the enclosing method, truncated to length $k$:

$$o_{ij\cdots pq} \text{ represents the object created by } s_q \text{ when the enclosing method ran on } o_{ij\cdots p}.$$

Going back to the encapsulation example: with object sensitivity, `set` is analysed twice, once with context $\{o_1\}$ and once with context $\{o_2\}$. The first instance only writes `oA` into `this.f`, the second only writes `oB`. The points-to picture comes out as intended.

### 3.3 Parameterization

The paper exposes two parameters. The naming depth $k$ controls how deep the heap context sequence goes, and may in principle be chosen per allocation site (so a container's backing array could use $k = 2$ while ordinary objects use $k = 1$). A second parameter $R^\star$ controls which variables are replicated across contexts; setting $R^\star = \emptyset$ collapses to context insensitivity. Both of these parameters are orthogonal to the choice of receiver objects as contexts.

### 3.4 Constraint rules

The constraint rules are still recognisably Andersen. Throughout this subsection, $v$ ranges over the reference-typed variables of the program: locals of any method, formal parameters, `this`, and return variables. Each variable now carries a context: $pt(v^c)$ is the set of objects $v$ may point to when its enclosing method is analysed in calling context $c$. The intra-procedural rules from Section 2 thread the context through unchanged. For example, the assign rule becomes

$$v_2 = v_1 \implies pt(v_1^c) \subseteq pt(v_2^c),$$

and similarly for new, load, and store. Both sides live in the same context $c$ because intra-procedural assignments happen within one method run.

The only rule that changes shape is the virtual call rule. When resolving `l = r0.m(r1, ..., rn)` in caller context $c$, the callee's calling context $c'$ is taken to be the receiver object itself: for each $o \in pt(r_0^c)$, the analysis sets $c' = o$, dispatches to $m' = \textsc{lookup}(\mathit{type}(o), m)$, and adds the constraints

$$\begin{aligned} o &\in pt(\mathit{this}_{m'}^{c'}) \\ pt(r_i^c) &\subseteq pt(p_{i\,c'}^{m'}) \quad \text{for each } i = 1, \ldots, n \\ pt(\mathit{ret}_{m'}^{c'}) &\subseteq pt(l^c) \end{aligned}$$

Compare this against the Andersen virtual call rule in Section 2: the shape is identical, only the analysis is now keyed on $(v, c)$ rather than $v$, and the callee's context $c'$ is set to the receiver $o$ rather than being absent.

### 3.5 Incomparability with call site sensitivity

Object sensitivity is not comparable to call site sensitivity. There are programs where call site sensitivity is strictly more precise, and there are programs where object sensitivity is strictly more precise. They are formally incomparable. What the paper argues is that on real Java code, the second class of programs outnumbers the first, which is borne out by their experiments.

**Final Conclusion** The shift from call site sensitivity to object sensitivity is largely a shift in what counts as identification. Call site sensitivity identifies a method run by the path of calls that led to it, which is a property of the syntactic call graph. Object sensitivity identifies a method run by the heap object whose method was called, which is a property of the abstract heap that the analysis is itself building. Call site sensitivity suits programs that distinguish runs of a function, and object sensitivity suits programs that distinguish runs of a method on different state. Java programs spend most of their time in the latter mode, since dynamic dispatch routes control through receiver objects rather than through call sites, which is why object sensitivity dominates for Java.

## 4 Picking Contexts

Six years later, Smaragdakis et al. (2011) revisit the design space and argue that prior work has been treating context sensitivity as a single labelled choice rather than as a parameterised one.

### 4.1 record and merge

The paper argues that the actual content of any context sensitive analysis is in two functions, RECORD and MERGE. To make this precise the framework distinguishes two kinds of context, attached to two different kinds of thing in the program:

- the *calling context* $c \in C$ qualifies the analysis of methods and their local variables (it controls how many times a method is re-analysed and which run is which)
- the *heap context* $hc \in HC$ qualifies abstract heap objects (it controls how many distinct abstract objects come from a single `new` statement)

RECORD and MERGE are the only places where these two kinds of context get constructed. Their signatures are

$$\textsc{record} : \text{AllocSite} \times C \to HC, \qquad \textsc{merge} : \text{CallSite} \times HC \times C \to C.$$

RECORD is invoked at every allocation site. When the analysis sees a `new T()` happening inside a method that is being analysed in calling context $c$, it asks RECORD(site, $c$) for the heap context to attach to the new abstract object. The full name of the abstract object becomes (allocation site, heap context). If RECORD ignores its arguments and always returns the empty heap context, no heap cloning happens. If RECORD returns $c$ itself, heap cloning happens and the heap context equals the calling context.

MERGE is invoked at every method call. When the analysis sees a call inside a method analysed in caller context $c$, with a receiver object whose heap context is $hc$, at call site $l$, it asks MERGE$(l, hc, c)$ for the calling context $c'$ to give the callee. The callee's locals, parameters, `this`, and return variable then live under this $c'$. Two calls that produce the same $c'$ share a single analysis of the callee; two calls that produce different $c'$ get analysed separately. So MERGE is what decides how method analyses are partitioned across calls.

The reframing the paper insists on is that each named flavour of context sensitivity is exactly one choice of MERGE:

- **Call site sensitivity (1-CFA):** MERGE$(l, hc, c) = l$. Use the call site label as the new context, ignore the receiver and the caller's context.
- **Object sensitivity (Milanova et al. (2005)'s original):** MERGE$(l, hc, c) = hc$. Use the receiver's heap context as the new context, ignore the call site and the caller's context.
- **Type sensitivity (Introduced in this paper):** MERGE$(l, hc, c) = T(hc)$, where $T$ extracts a class from a heap context.

Two analyses that both call themselves "object sensitive" can disagree on the exact MERGE they use, and the resulting precision and cost can differ a lot.

The authors point out a specific instance of this divergence. The original Milanova et al. (2005) object sensitivity sets MERGE$(l, hc, c) = hc$: the callee's calling context is *exactly* the receiver's heap context, with nothing else mixed in. The convention in the PADDLE implementation and several follow-ups is a different MERGE: it concatenates the receiver's allocation site with elements of the caller's context $c$, so the callee's calling context carries both receiver and caller information rather than receiver information alone. The paper calls these two configurations *full* object sensitivity (Milanova et al. (2005)'s choice) and *plain* object sensitivity (PADDLE's choice). They have the same depth, but they classify program states differently. The paper shows empirically that full is both more precise and often faster than plain.

### 4.2 Type sensitivity

The paper's second contribution is type sensitivity, a coarsening of object sensitivity. The idea is to use a class as the context element instead of an allocation site. The detail that matters is which class. There are two natural candidates:

1. the dynamic type of the allocated object, or
2. the static type of the class that *encloses* the allocation site, that is, the class of the allocator.

The paper argues that choice (1) is a bad design: the dynamic type of an object is already pinned down by virtual dispatch, so reusing it as a context element adds almost no new partitioning. Choice (2) is what the paper recommends. The class of the allocator is loosely correlated with the call site and the receiver, so it acts as a useful new dimension.

The practical consequence is that type sensitive contexts are far smaller than allocation site contexts (the number of classes in a program is typically much smaller than the number of

allocation sites), so the cost of context sensitivity drops sharply, while precision typically stays close to full object sensitivity at the same depth.

### 4.3 Context as a classifier

The principle the authors derive from this is that a good context is an *uncorrelated* context. The role of a context is to partition the universe of program states into uniform equivalence classes. A context element that is highly correlated with information the analysis already has (the dynamic type of an already type-resolvable object) wastes its partitioning budget on distinctions that did not need making. A context element that is loosely correlated with what the analysis already has (the type of the allocator) gives useful new information. This reframes context choice as a classifier design problem rather than a depth tuning exercise.

### 4.4 Taxonomy

The paper introduces a uniform notation for configurations of the form $X\mathrm{n} + \mathrm{m}H$, where $X$ is the context kind (`plain`, `full`, `type`), $n$ is the calling-context depth, and $m$ is the heap-context depth. So `1obj+H` is one-object sensitivity with a one-deep heap context, `2type+1H` is depth-2 type sensitivity with a one-deep heap context, and so on.

## 5 Selective Context Sensitivity

Li et al. (2018) ask the next question: where in the program should context sensitivity be applied at all? Conventional analyses are uniform, every method gets analysed under multiple contexts, but for a lot of methods the extra contexts are redundant. The wasted work is the dominant scalability bottleneck.

The strategy of the paper is to identify, before the main analysis runs, which methods are *precision critical*: those whose extra contexts would actually change the answer. The pipeline has three parts. First, run a cheap context insensitive pre-analysis to learn the program's coarse points-to facts. Second, characterise the routes by which imprecision actually propagates as data-flow patterns within a single class. Third, build a flow graph that turns those patterns into reachability paths, and apply context sensitivity only to the methods that lie on a precision-relevant path. The next subsection lays out the three patterns concretely; the one after introduces the graph; the last assembles the full pipeline.

### 5.1 Three precision-loss patterns

The structural claim of Li et al. (2018) is that most of the context insensitive imprecision that context sensitivity could fix fits into three patterns of data flow within a single class. The patterns are framed in terms of the methods of a class:

**Definition 1** (In/Out methods)**.** *A method $m$ of a class $C$ is an* In *method of $C$ if it has at least one parameter, and an* Out *method of $C$ if its return type is non-void. A method may be both.*

Each precision-loss pattern flows from an In method of a class $C$ to an Out method of the same class.

**Direct flow:** The setter / getter pattern.

```
class MyObj {
    Object id;
    void set(Object x) { this.id = x; }
    Object get() { return this.id; }
```

```
}

void main() {
    MyObj m1 = new MyObj(); m1.set(new X());
    MyObj m2 = new MyObj(); m2.set(new Y());
    Object x = m1.get();    // would like: only X
    Object y = m2.get();    // would like: only Y
}
```

A context insensitive analysis sees one summary of `set` and one of `get`, and ends up reporting that both `x` and `y` can be either `X` or `Y`. Object sensitivity on `MyObj` fixes this. The flow that triggers the fix is the path from the parameter of `set` (the In method) to the return of `get` (the Out method) within `MyObj`.

**Wrapped flow:** The collection / iterator pattern.

```
class MyObj {
    Object item;
    void put(Object o) { this.item = o; }
    MyObjWrapper wrap() { return new MyObjWrapper(this.item); }
}
class MyObjWrapper {
    Object src;
    MyObjWrapper(Object o) { src = o; }
    Object unwrap() { return this.src; }
}

void main() {
    MyObj m1 = new MyObj(); m1.put(new X());
    MyObj m2 = new MyObj(); m2.put(new Y());
    MyObjWrapper w1 = m1.wrap();
    MyObjWrapper w2 = m2.wrap();
    Object x = w1.unwrap();    // would like: only X
    Object y = w2.unwrap();    // would like: only Y
}
```

The path from the parameter of `put` to the return of `wrap` goes from a store into the receiver's field (`this.item = o`) to a fresh `MyObjWrapper` whose `src` field receives that element, and the new wrapper flows out as the return value. Under a context insensitive analysis, `put` and `wrap` each have a single summary, so both `X` and `Y` flow into `this.item` of every `MyObj`, and into the `src` field of every `MyObjWrapper`. The `unwrap` call then reads `this.src` from the merged wrapper, so both `x` and `y` come out as $\{\texttt{X}, \texttt{Y}\}$ instead of the intended singletons.

**Unwrapped flow:** The synchronized-set or boxed-getter pattern.

```
class MyObj {
    Object item;
    void put(Object o) { this.item = o; }
    Object getItem() { return this.item; }
}
class MyObjWrapper {
    MyObj inner;
    MyObjWrapper(MyObj m) { this.inner = m; }
    Object getItem() { return this.inner.getItem(); }
}

```

```
void main() {
    MyObj m1 = new MyObj(); m1.put(new X());
    MyObj m2 = new MyObj(); m2.put(new Y());
    MyObjWrapper w1 = new MyObjWrapper(m1);
    MyObjWrapper w2 = new MyObjWrapper(m2);
    Object x = w1.getItem();   // would like: only X
    Object y = w2.getItem();   // would like: only Y
}
```

The path from the parameter of the `MyObjWrapper` constructor to the return of `MyObjWrapper.getItem` goes through a store into `this.inner` followed by a delegating call `this.inner.getItem()` that loads `item` out of the inner `MyObj`, and that loaded object flows out as the return value. The load is the unwrap: a different object (`item`) is pulled out of an enclosing one (`inner`). Under a context insensitive analysis, the constructor, `put`, and both `getItem` methods each have a single summary, so `this.inner` of every `MyObjWrapper` ends up pointing to both `m1` and `m2`, and the `item` field of every `MyObj` ends up holding both `X` and `Y`. So both `x` and `y` come out as $\{\texttt{X}, \texttt{Y}\}$ instead of the intended singletons.

### 5.2 Object Flow Graph and Precision Flow Graph

To detect the three patterns mechanically, Li et al. (2018) build a flow graph from the pre-analysis. The basic graph is the Object Flow Graph (OFG), whose nodes are program pointers (variables and abstract object fields) and whose edges express direct may-flow induced by assignments, parameter passing, and field load and store. Reachability in the OFG corresponds to the question "can object $o$ reach pointer $v$" The OFG by itself captures direct flow, but it does not, on its own, expose the wrapping and unwrapping that drive the other two patterns; for those, an additional construction is needed.

**Definition 2** (Wrapping and unwrapping)**.** *An object $O$ is* wrapped *into another object $W$ when $O$ is stored into a field of $W$. An object $O$ is* unwrapped *from $W$ when $O$ is loaded from a field of $W$.*

The Precision Flow Graph (PFG) extends the OFG with extra edges, one per wrapping or unwrapping operation in the program. The construction (Algorithm 1 in the paper, PfgBuilder) starts from each parameter of each In method of class $C$ and traverses the OFG. At every variable node $b$ it reaches, it inspects the surrounding statement and adds the wrap/unwrap edge accordingly:

- for each load `b = a.f` encountered along the traversal, add an unwrap edge $a \to b$;
- for each store `b.f = a` encountered along the traversal, add a wrap edge from $a$ to the variable that originally allocated the abstract object pointed to by $b$, for every such object in $pt(b)$.

The PFG is built per class. Once it is in place, all three precision-loss patterns, and any combinations of them, become paths in a single graph: direct flow as a chain of OFG edges, wrapped flow as a chain ending in a wrap edge, and unwrapped flow as a chain that traverses an unwrap edge. So the question "does method $m$ of class $C$ lie on a precision-relevant flow?" is reduced to a graph reachability question on the per-class PFG.

### 5.3 Algorithm

The pipeline is:

1. Run a context insensitive pre-analysis to get $pt(\cdot)$.

2. Build the OFG, then for each class augment it into the PFG by adding wrap and unwrap edges keyed off $pt(\cdot)$.
3. For each class, run a forward reachability through the PFG from the parameters of *In* methods. Methods whose locals or stored objects lie on a path to the return of an *Out* method are marked *precision critical*.
4. Run the main context sensitive analysis, but apply context sensitivity only to precision critical methods.

# 6 LSRV-contexts

Thakur and Nandivada (2019) introduce LSRV-contexts (Level-Summarized Relevant Value-contexts) as a more scalable replacement for the value-contexts of Padhye and Khedker (2013).

## 6.1 Value-contexts

The starting point is the value-context abstraction, introduced by Khedker and Karkare (2008) and applied to whole-program points-to analysis by Padhye and Khedker (2013). A value-context keys the analysis of a method by the dataflow value at the call site, that is, by the parameter-reachable points-to subgraph the method receives, rather than by the call-string that led to the call. Two calls with the same input state share a context; two calls with different input states get separate analyses.

The motivation is that value-contexts bound the unbounded growth of call-strings while preserving their precision. Under $k$-CFA, the context space grows with the depth of the call stack and is conservatively truncated to length $k$, which loses precision. Under value-contexts, the context space is the lattice of input states, which is finite for set-union over allocation sites: two distinct call-strings that pass the same dataflow value collapse into the same context, and the analysis runs once for that input regardless of how many syntactically distinct paths reach it.

## 6.2 Why value-contexts do not scale

Thakur and Nandivada (2019) point out two bottlenecks of plain value-contexts on whole-program heap analyses:

- **Too much comparison** At every call site the new context must be compared against every previously seen context of the callee for exact equality on whole heap subgraphs. Subgraph comparison is expensive in time and memory.
- **Too many contexts** Many distinct subgraphs are generated, each triggering a re-analysis and storing its own summary. The cost multiplies through callee chains.

The corresponding observations the paper builds on are: the whole parameter-reachable subgraph is usually not relevant to what the callee actually reads or writes; and many methods do not modify the caller's heap at all, so their re-analysis can be deferred without affecting caller summaries.

## 6.3 Compare Less

The first technique trims and summarises the context. For each method $m$ and each parameter $p_i$, a pre-analysis computes the access depth $mInfo_m(i)$: the maximum depth of fields read or written through $p_i$ inside $m$. The relevant value-context is the parameter-reachable subgraph truncated to that depth.

The relevant subgraph is then summarised level by level. At each level, all heap nodes reachable through the same field path are collapsed into a single abstract node carrying a summary value drawn from a finite analysis-specific lattice (escape statuses for thread-escape analysis; types for control-flow analysis). The result, the LSRV-context, is a coarsening of the relevant value-context.

Because the access depth is bounded per method and the per-node abstract value lives in a finite lattice, the LSRV lattice is finite by construction. LSRV-contexts therefore terminate at any finite parameterisation, including in combinations such as heap cloning where ordinary value-contexts do not.

### 6.4 Defer More

The second technique avoids re-analysing methods whose results do not affect any caller. A method $m$ is *caller-ignorable* if its access depth on every parameter is zero, i.e., $m$ neither reads nor writes the caller's heap. Such methods are recorded in a deferred-methods map during the main analysis and re-analysed only after the main analysis finishes, in a post-analysis pass that does not need to merge their summaries back into caller heaps.

### 6.5 The three-phase pipeline

1. **Pre-analysis** A flow-insensitive bottom-up pass over the call graph that computes *mInfo* for every method and parameter.
2. **Main analysis** A flow- and context-sensitive points-to analysis that, at every call, defers the call if the callee is caller-ignorable and otherwise compares the new context to previously seen contexts via level-summarised equality, reusing any matching summary.
3. **Post-analysis** Re-runs the main analysis on each deferred method in each context recorded for it.

**Final Conclusion** LSRV trims value-contexts in two ways: the relevant-and-truncated step keeps only the parameter-reachable subgraph up to access depth, and the level-summary step collapses each level into a finite analysis-specific lattice. Caller-ignorable methods are deferred and analysed once after the main analysis. The combined effect is that the context lattice becomes finite at every depth, and re-analysis cost drops, while precision matches that of full value-contexts.

## 7 Mixing Contexts

Thakur and Nandivada (2020) address a complementary question. Where Li et al. (2018) ask where in the program to apply a fixed context abstraction, Thakur and Nandivada (2020) ask which abstraction to apply, and propose using two at once.

### 7.1 The premise

Since Milanova, it has been understood that object sensitivity and call site sensitivity are formally incomparable: each catches optimisation opportunities the other misses. The empirical practice of picking one and discarding the other has therefore always been a precision compromise.

The two abstractions partition method runs along orthogonal axes. Call site sensitivity distinguishes runs of a method by where in the program they were invoked from; object sensitivity distinguishes them by which heap object the call was made on. Whenever a single receiver participates in calls from multiple distinct call sites with different arguments, only call site

sensitivity separates those calls; object sensitivity collapses them into the receiver's single context and merges their argument flows. Whenever a single call site dispatches on a receiver that could be one of several distinct allocations, only object sensitivity separates the cases; call site sensitivity collapses them into the call site's single context and merges their receiver flows. Each abstraction therefore catches optimisation opportunities the other misses, and neither dominates the other.

This compromise has held because of scalability. Call site sensitive analyses with heap cloning at full call-string length (`kcsH` with $k = \infty$) are combinatorially expensive, and `valcsH` may not even terminate under recursion. Recent work on more scalable formulations of call site sensitivity, in particular the value-contexts of Padhye and Khedker (2013) and the authors' own LSRV-contexts (§6), finally make it feasible to combine the two.

### 7.2 LSRV with heap cloning

The starting point is `lsrvH`: LSRV-contexts (§6) together with heap cloning. The combination of plain value-contexts with heap cloning, `valcsH`, is non-terminating in general, because every recursive call generates a fresh heap-cloned object and hence a new value-context. `lsrvH` avoids this: as long as the analysis-specific lattice is finite (escape statuses for thread-escape analysis, types for control-flow analysis), the LSRV lattice itself is finite, and termination holds even with heap cloning at any finite $k$.

### 7.3 The mixed abstraction `lsrvkobjH`

The paper sets out two design criteria for mixing two abstractions: their optimisation opportunities should be non-overlapping (so the mix actually adds something), and the components should be as scalable as possible. The two picks follow from these criteria. From the call-site-sensitive family, `lsrvH` offers the best practical precision-scalability trade-off, as established in §6. From the object-sensitive family, `kobjH` is the standard scalable choice in the literature.

The mixed abstraction `lsrvkobjH` is built point by point. At each call site, a new context is created if either of the two component contexts has changed. The new context space is a subset of the cross-product of the two component spaces, and is finite because LSRV is finite and there are finitely many allocation sites. By construction, `lsrvkobjH` is at least as precise as either of its components on its own.

### 7.4 The receiver-store optimisation

A direct cross-product would not scale, even with LSRV. The paper's central observation is Insight 1: if a method $m$ and its callees do not store any locally allocated object into a non-primitive field of the receiver, then a change in the receiver object cannot change $m$'s summary, and so the `kobj` component of the mixed context is redundant for $m$. The pre-analysis computes a per-method `receiverStore` flag bottom-up on the call graph, identifying methods that satisfy Insight 1. At a call to a method that satisfies the insight, only the `lsrvH` component is constructed and compared; at a call to a method that does not, the full `lsrvkobjH` context is built. The cost of mixing is therefore concentrated on the methods where the receiver actually matters.

### 7.5 The precision lattice

The paper closes by placing the new abstraction in a precision lattice (Figure 7). On the call-string side, `kcsH` at full call-string length is the most precise but does not scale; `valcsH` may not terminate under recursion with heap cloning; `lsrvH` is the most precise of the call-string-based abstractions that scales reliably. On the object-sensitivity side, `kobjH` sits above `kobj`. The

new `lsrvkobjH` strictly dominates both `lsrvH` and `kobjH` in precision; in theory it remains incomparable with `kcsH`. With the `receiverStore` optimisation, `lsrvkobjH` stays scalable on the benchmarks where a uniform mix would not finish.

## References


Uday P Khedker and Bageshri Karkare. Efficiency, precision, simplicity, and generality in interprocedural data flow analysis: Resurrecting the classical call strings method. In *International Conference on Compiler Construction*, pages 213–228. Springer, 2008.

Yue Li, Tian Tan, Anders Møller, and Yannis Smaragdakis. Precision-guided context sensitivity for pointer analysis. *Proceedings of the ACM on Programming Languages*, 2(OOPSLA):1–29, 2018.

Ana Milanova, Atanas Rountev, and Barbara G Ryder. Parameterized object sensitivity for points-to analysis for java. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 14(1):1–41, 2005.

Rohan Padhye and Uday P Khedker. Interprocedural data flow analysis in soot using value contexts. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on State of the Art in Java Program analysis*, pages 31–36, 2013.

Yannis Smaragdakis, Martin Bravenboer, and Ondrej Lhoták. Pick your contexts well: understanding object-sensitivity. In *Proceedings of the 38th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 17–30, 2011.

Manas Thakur and V Krishna Nandivada. Compare less, defer more: Scaling value-contexts based whole-program heap analyses. In *Proceedings of the 28th International Conference on Compiler Construction*, pages 135–146, 2019.

Manas Thakur and V Krishna Nandivada. Mix your contexts well: Opportunities unleashed by recent advances in scaling context-sensitivity. In *Proceedings of the 29th International Conference on Compiler Construction*, pages 27–38, 2020.